

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

08 - Árvores binárias de busca

Prof. Alex Kutzke

Junho de 2021

Slides adaptados do material produzido
pelo Prof. Rodrigo Minetto

Roteiro

Motivação e conceito

Operações

Operação de remoção

Exercícios propostos

Motivação

- Algoritmo de busca binária:
 - Bom desempenho para busca;
 - Mas apenas para conjuntos ordenados;
 - Necessita de estrutura de dados com indexação direta;
- Inserção e remoção em vetores ordenados:
 - Necessita rearranjar elementos;

Motivação

- É possível uma estrutura de dados tal que
 - Inserção eficiente;
 - Remoção eficiente;
 - Busca eficiente;
 - Tamanho flexível;
- ???

Motivação

- É possível uma estrutura de dados tal que
 - Inserção eficiente;
 - Remoção eficiente;
 - Busca eficiente;
 - Tamanho flexível;
- ???
- Sim!

Motivação

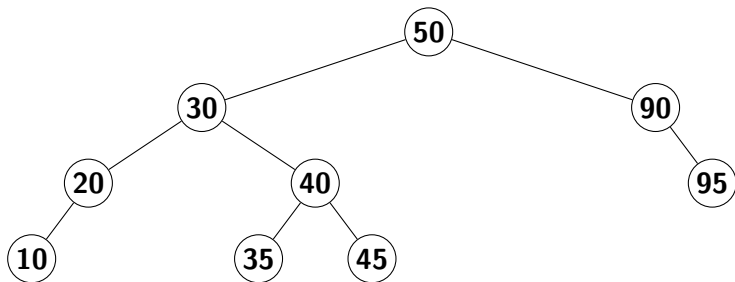
- É possível uma estrutura de dados tal que
 - Inserção eficiente;
 - Remoção eficiente;
 - Busca eficiente;
 - Tamanho flexível;
- ???
- Sim!
- Árvores binárias de busca são um exemplo;

Árvores binárias de busca

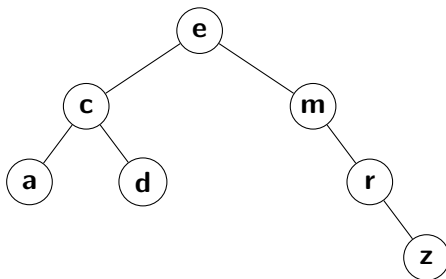
- **Árvores binárias de busca** são árvores binárias com uma **propriedade fundamental**:

O valor associado à **raiz** é sempre **maior** do que o valor associado a qualquer nó da subárvore à **esquerda** e é sempre **menor** do que o valor associado a qualquer nó da subárvore à **direita**.

Exemplos



Exemplos



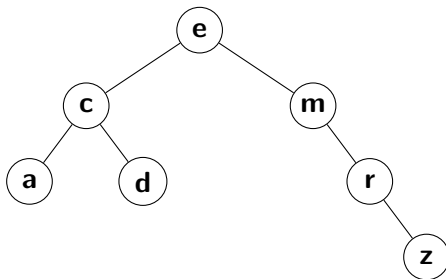
Percurso in-ordem

- A propriedade fundamental garante que:

Quando percorrida em ordem **simétrica**, ou seja, in-ordem (E,R,D), os valores são encontrados em ordem crescente.

Percurso in-ordem: exemplo

In-ordem: a,c,d,e,m,r,z.



Percurso in-ordem

- A propriedade fundamental **também** possibilita que:

As operações de **busca, inserção e remoção** podem ser realizadas de forma **eficiente**. **Como?**

Percurso in-ordem

- A propriedade fundamental **também** possibilita que:

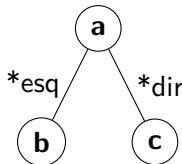
As operações de **busca, inserção e remoção** podem ser realizadas de forma **eficiente**. **Como?**

- Basta aplicar o mesmo algoritmo da busca binária para todas as operações acima.

Estrutura

- Podemos utilizar a mesma estrutura de dados apresentada na última aula;
- Árvores binária de busca são árvores binárias normais mas com uma propriedade fundamental.

```
1 typedef struct arvore {  
2     int info;  
3     struct arvore *esq;  
4     struct arvore *dir;  
5 } Arvore;
```



Busca

- A operação de **busca** explora diretamente a propriedade fundamental da árvore:

```
1  int buscar (Arvore *a, int v) {
2      if (a == NULL) { return 0; } /*Nao achou*/
3      else if (v < a->info) {
4          return buscar (a->esq, v);
5      }
6      else if (v > a->info) {
7          return buscar (a->dir, v);
8      }
9      else { return 1; } /*Achou*/
10 }
```

Inserção

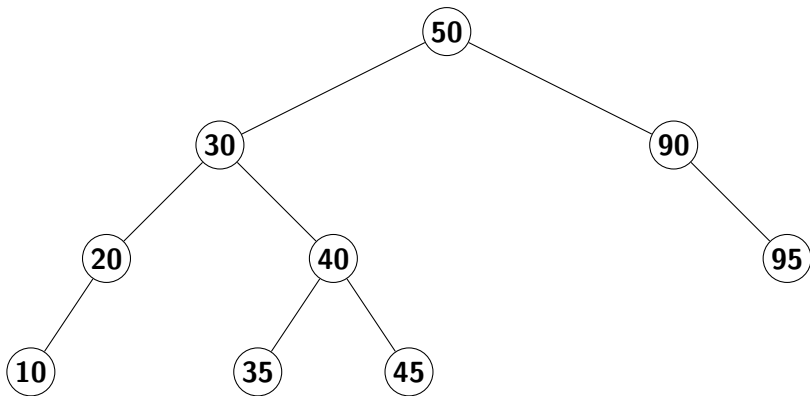
- A operação de **inserção** precisa garantir que propriedade fundamental da árvore seja mantida;
- Ou seja, o novo elemento precisa ser inserido na **posição correta**;
- A solução mais simples é **sempre inserir elementos folha**;
- Para isso seguimos o seguinte algoritmo recursivo para inserir um novo elemento x :
 1. Se x é menor que a raiz, insere na subárvore da esquerda;
 2. Se x é maior que a raiz, insere na subárvore da direita;
 3. Se a árvore é vazia, apenas cria um novo nó com valor x .

Inserção: implementação

```
1  Arvore* inserir (Arvore *a, int v) {
2      if (a == NULL) {
3          a = (Arvore*)malloc(sizeof(Arvore));
4          a->info = v;
5          a->esq = a->dir = NULL;
6      }
7      else if (v < a->info) {
8          a->esq = inserir (a->esq, v);
9      }
10     else { a->dir = inserir (a->dir, v); }
11     return a;
12 }
```

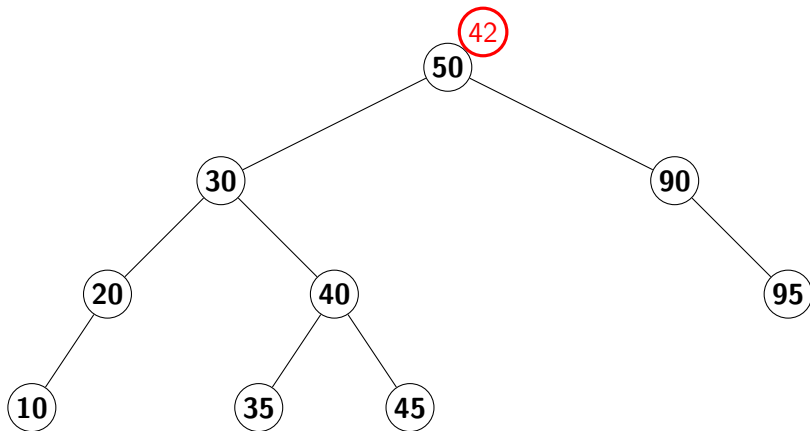
Exemplo inserção

Inserir o elemento 42.



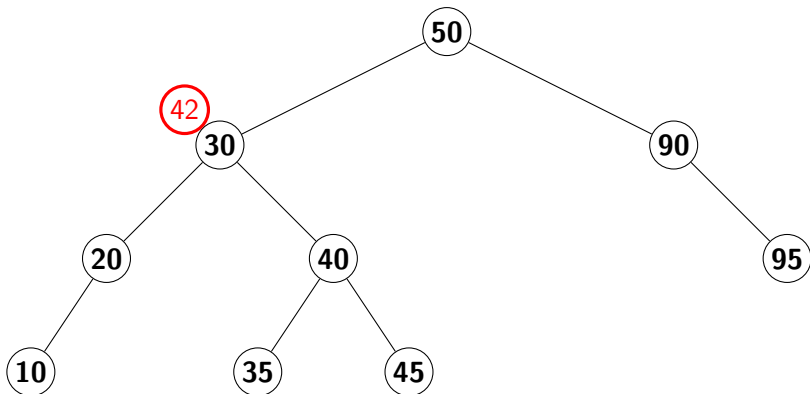
Exemplo inserção

Inserir o elemento 42.



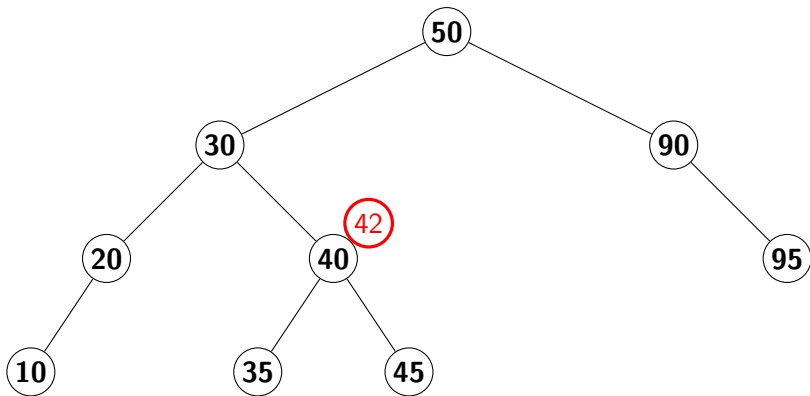
Exemplo inserção

Inserir o elemento 42.



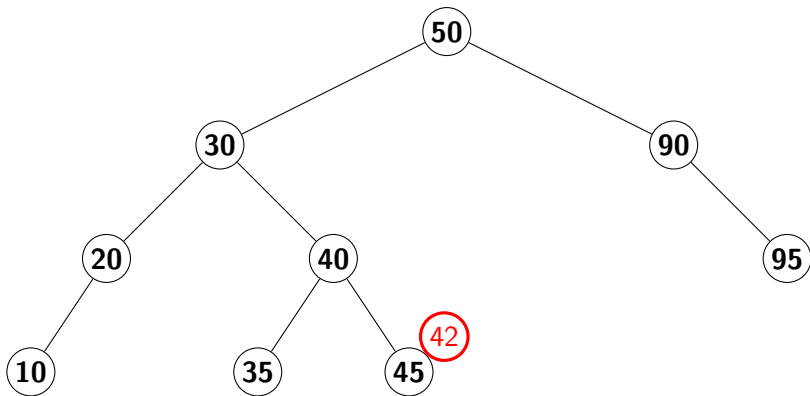
Exemplo inserção

Inserir o elemento 42.



Exemplo inserção

Inserir o elemento 42.



Operação: remoção

A operação de **remoção** é mais complexa que a inserção. Novamente, a implementação dessa função vem da natureza recursiva da árvore:

- Se a árvore for **vazia**, nada tem de ser feito, pois o elemento não está presente na árvore;
- Se a árvore **não for vazia**:
 1. Compara-se o valor armazenado no nó raiz ao valor que se deseja retirar da árvore;
 2. Se o valor a ser removido for **menor** do que a raiz, chama-se a função recursivamente para subárvore da **esquerda**;
 3. Se o valor a ser removido for **maior** do que a raiz, chama-se a função recursivamente para subárvore da **direita**;
 4. Se o valor a ser removido for **igual** a raiz, remove-se esse elemento;
- Entretanto, teremos 3 casos diferentes para esse operação do item 4;

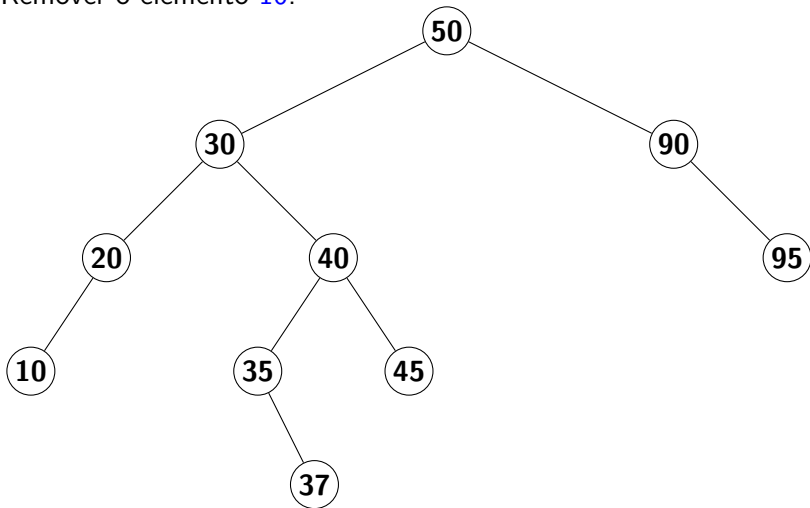
Operação: remoção (1º caso)

Três situações diferentes para a remoção de um nó na árvore binária de busca binária:

- 1º **caso**: Quando o nó a ser removido é uma **folha** da árvore;
 - Ou seja, uma raiz sem filhos;
 - Basta liberar a memória alocada para o elemento removido e retornar NULL para que a árvore pai possa atualizar seu estado;

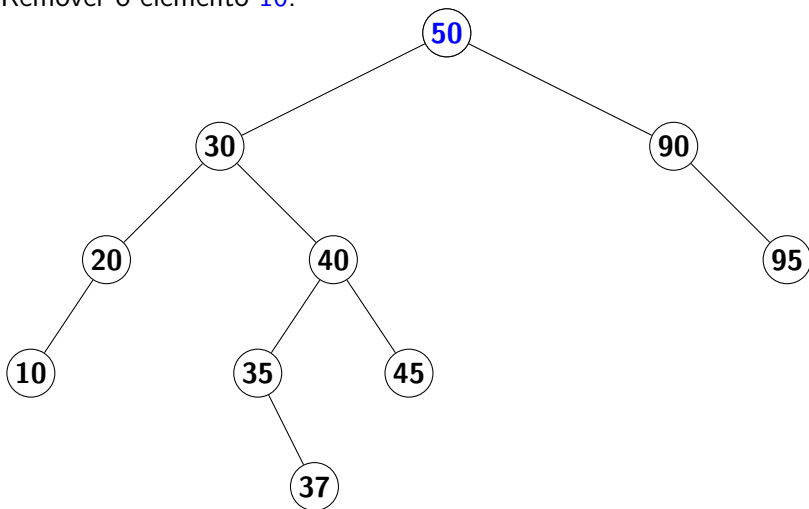
Exemplo remoção (1º caso)

Remover o elemento 10.



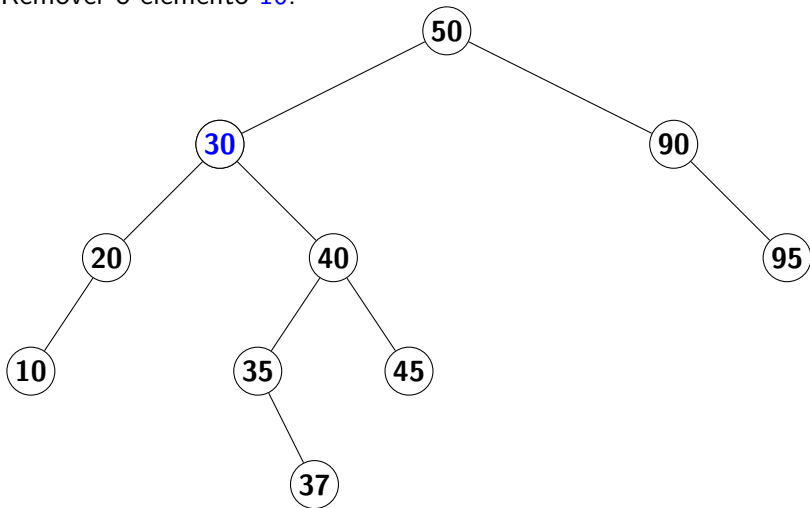
Exemplo remoção (1º caso)

Remover o elemento 10.



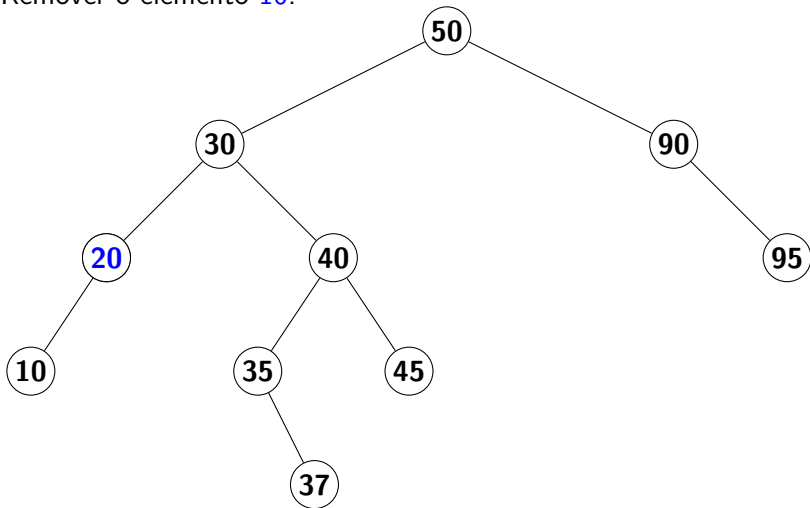
Exemplo remoção (1º caso)

Remover o elemento 10.



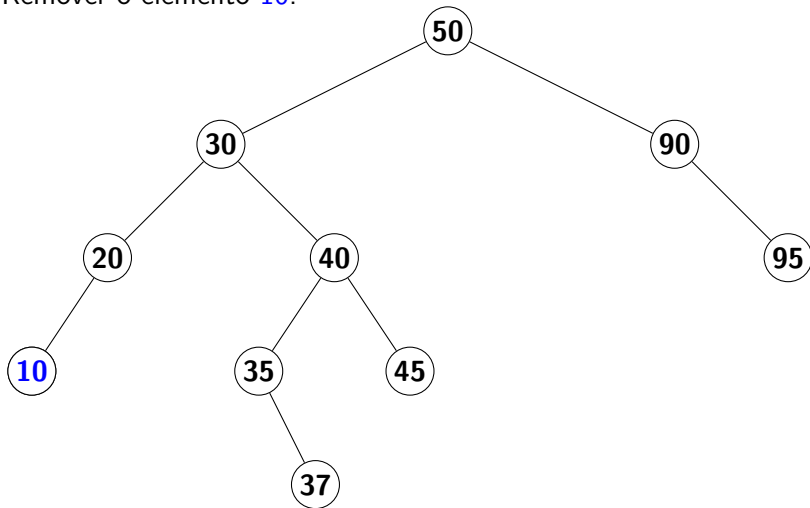
Exemplo remoção (1º caso)

Remover o elemento 10.



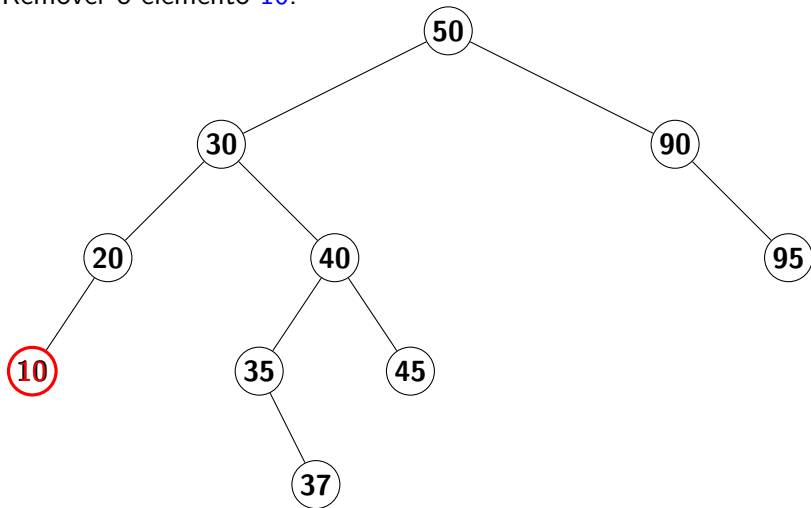
Exemplo remoção (1º caso)

Remover o elemento 10.



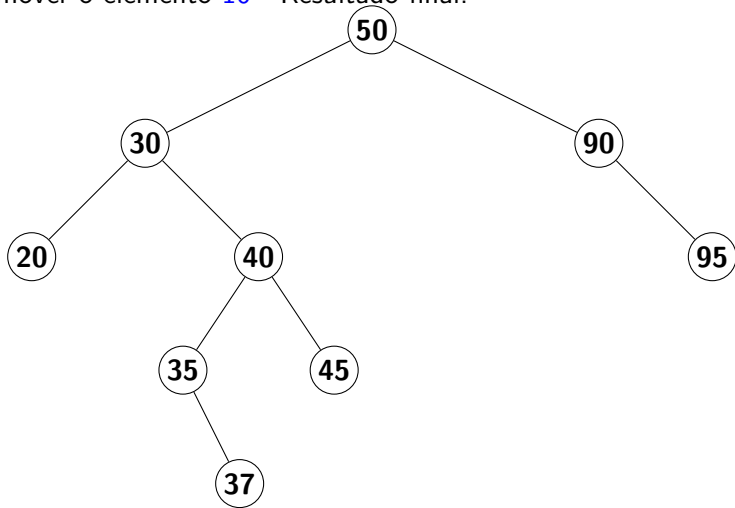
Exemplo remoção (1º caso)

Remover o elemento 10.



Exemplo remoção (1º caso)

Remover o elemento 10 - Resultado final:

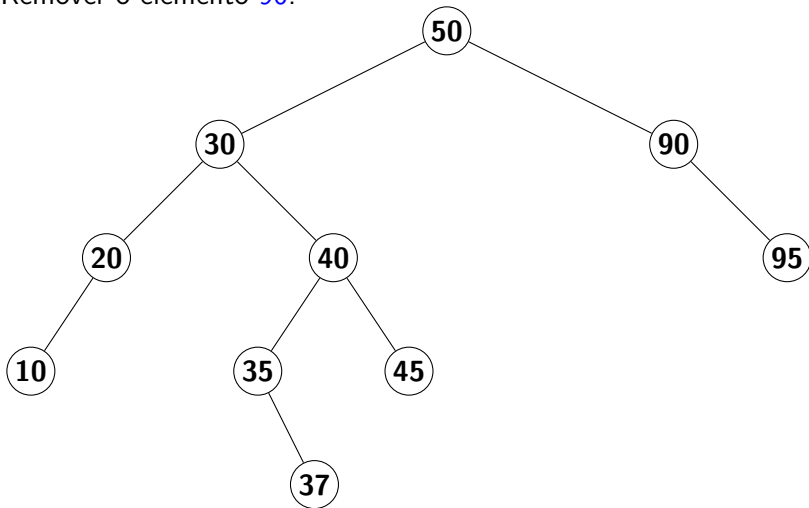


Operação: remoção (2º caso)

- 2º **caso**: Quando o nó a ser removido possui apenas **um filho**
 - Basta liberar a memória alocada para o elemento removido e retornar o ponteiro do único filho para que a árvore pai possa atualizar seu estado;

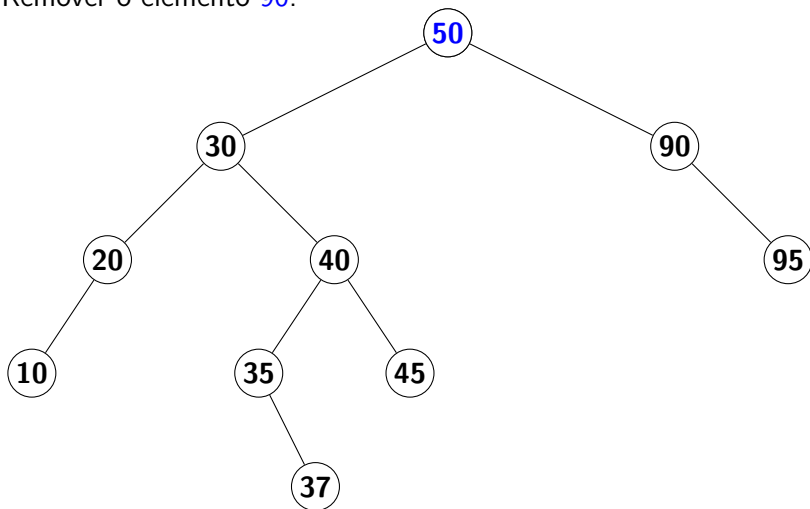
Exemplo remoção (2º caso)

Remover o elemento 90.



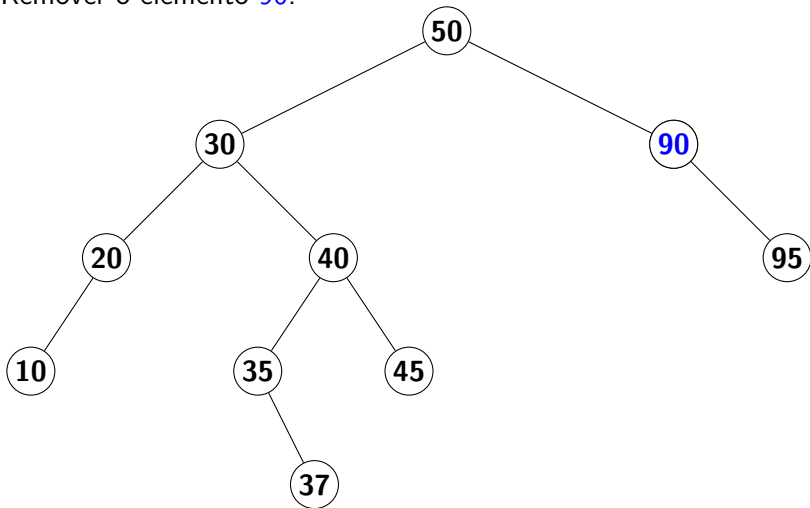
Exemplo remoção (2º caso)

Remover o elemento 90.



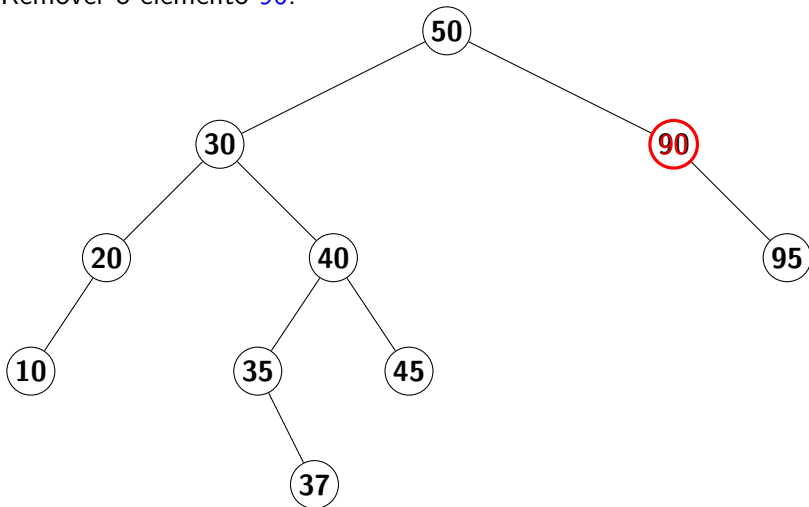
Exemplo remoção (2º caso)

Remover o elemento 90.



Exemplo remoção (2º caso)

Remover o elemento 90.



Operação: remoção (3º caso)

- 3º **caso**: Quando o nó a ser removido possui **dois filhos**
 - É necessário um procedimento mais elaborado para que a propriedade fundamental da árvore seja mantida após a remoção;
 - A pergunta chave é: **Qual elemento da árvore deve ocupar o lugar daquele a ser removido?**

Operação: remoção (3º caso)

- 3º **caso**: Quando o nó a ser removido possui **dois filhos**
 - É necessário um procedimento mais elaborado para que a propriedade fundamental da árvore seja mantida após a remoção;
 - A pergunta chave é: **Qual elemento da árvore deve ocupar o lugar daquele a ser removido?**

Resposta: Aquele que **imediatamente o precede** ou que **imediatamente o sucede** na ordenação dos elementos da árvore.

Operação: remoção (3º caso)

Escolheremos aquele que **imediatamente o precede**.

Ou seja, mais próximo daquele que se deseja remover e que seja menor do que este.

Em outras palavras ...

Operação: remoção (3º caso)

Escolheremos aquele que **imediatamente o precede**.

Ou seja, mais próximo daquele que se deseja remover e que seja menor do que este.

Em outras palavras ...

Encontrar o elemento **mais à direita** da **subárvore da esquerda** do elemento que se deseja remover.

Operação: remoção (3º caso)

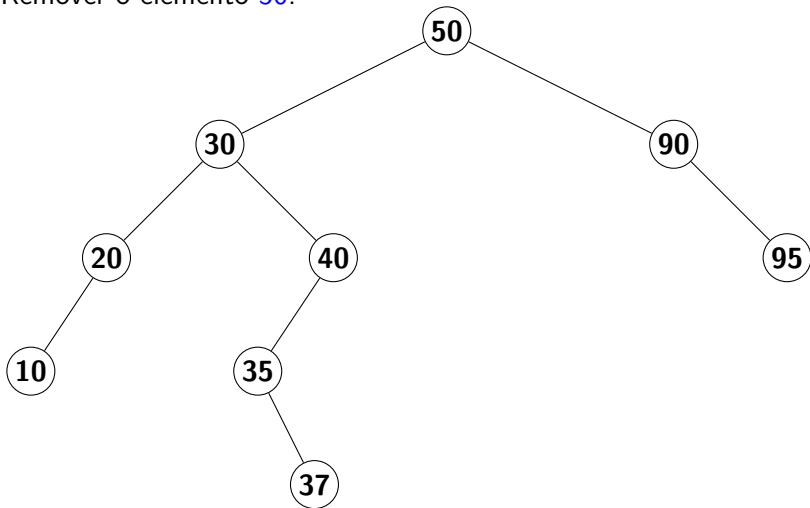
Assim, a remoção no terceiro caso segue os seguintes passos:

1. **Encontre** o elemento a ser removido;
2. Vá para sua **subárvore da esquerda**;
3. Desça até o **último nó mais à direita** dessa subárvore;
4. **Atribua** o valor desse nó ao nó onde se encontra o elemento a ser removido;
5. **Remova** esse último nó;

A remoção final será como nos casos 1 ou 2, pois ou ele será um nó folha ou um nó com apenas um filho (um filho à esquerda);

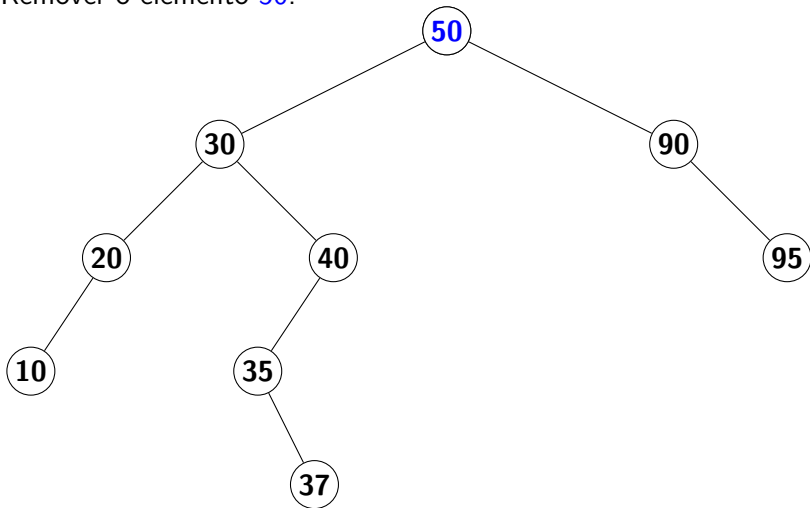
Exemplo remoção (3º caso)

Remover o elemento 50.



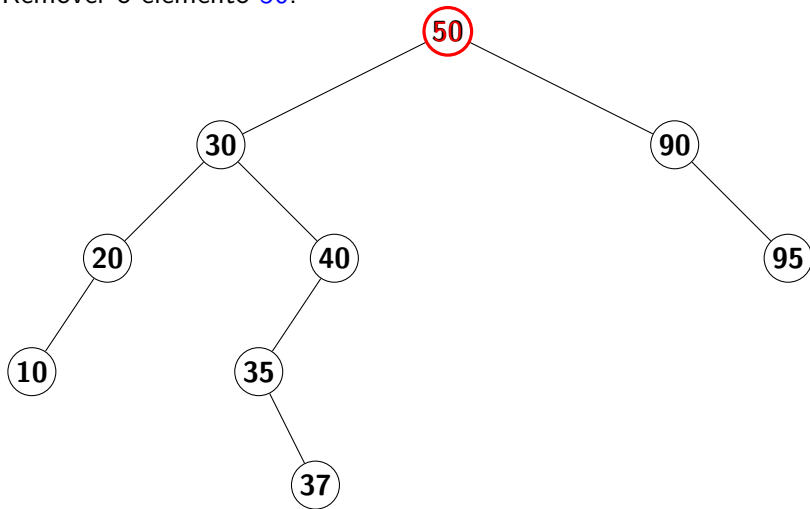
Exemplo remoção (3º caso)

Remover o elemento 50.



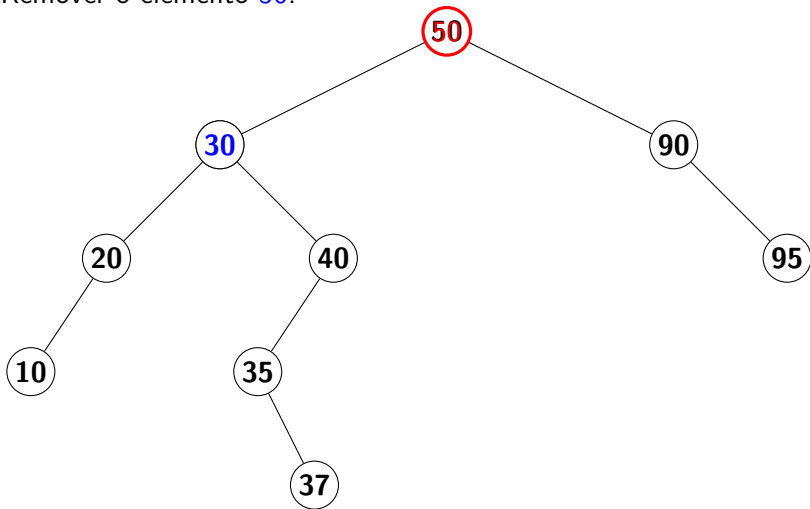
Exemplo remoção (3º caso)

Remover o elemento 50.



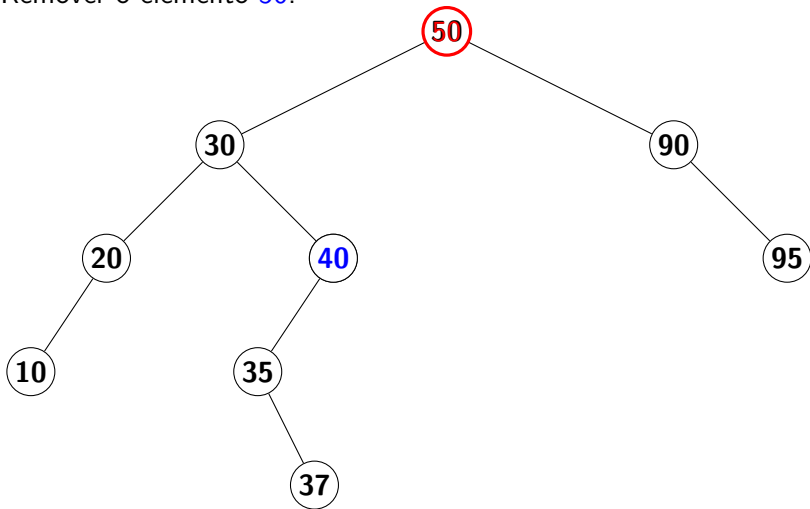
Exemplo remoção (3º caso)

Remover o elemento 50.



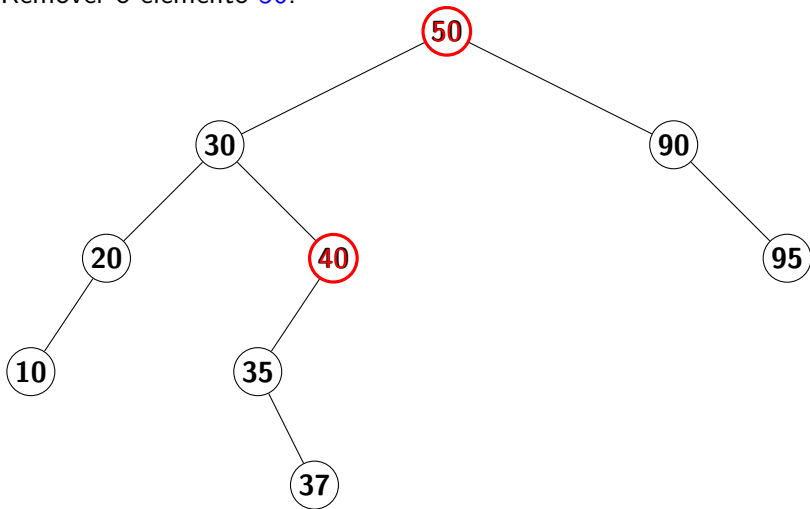
Exemplo remoção (3º caso)

Remover o elemento 50.



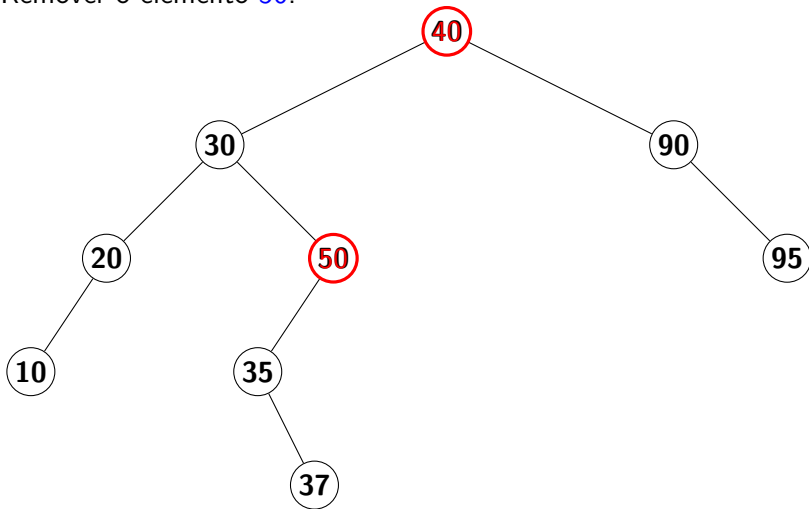
Exemplo remoção (3º caso)

Remover o elemento 50.



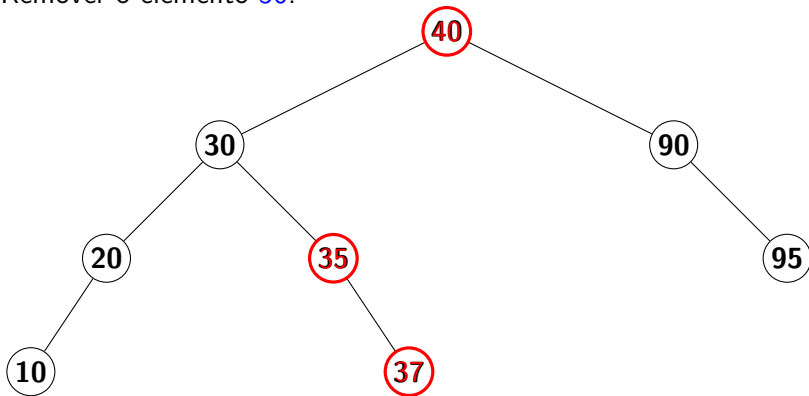
Exemplo remoção (3º caso)

Remover o elemento 50.



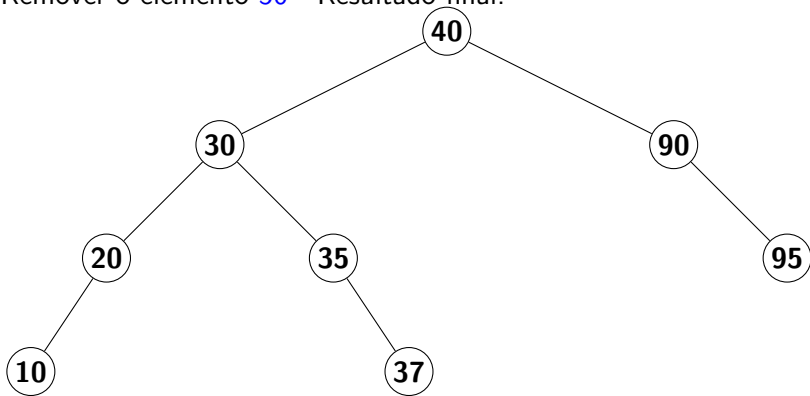
Exemplo remoção (3º caso)

Remover o elemento 50.



Exemplo remoção (3º caso)

Remover o elemento 50 - Resultado final:



Exercícios propostos

Exercício 1 Desenhe as árvores binárias de busca para os seguintes números inseridos nessa ordem:

- 1, 2, 3, 4, 5, 6, 7;
- 7, 6, 5, 4, 3, 2, 1;
- 4, 6, 3, 5, 1, 7, 3;

Qual das três é melhor para buscar um dado elemento.

Exercícios propostos

Exercício 2 Escreva um programa que produza 30 números aleatórios entre 0 e 999. Insira esses números em uma árvore binária de busca e imprima os valores na árvore (utilizando um percurso in-ordem). Os valores estão ordenados?

Exercício 3 Escreva um programa que produza 100000 números aleatórios entre 0 e 99999. Insira esses números em uma árvore binária de busca. Procure por um valor que não existe, por exemplo 100000. Quanto tempo a busca levou?

Exercícios propostos

Exercício 4 Escreva um programa que produza 100000 números em ordem (0 até 99999). Insira esses números em uma árvore binária de busca. Procure por um valor que não existe por exemplo 100000. Quanto tempo a busca levou?

Exercício 5 Escreva a função “min” que encontre uma chave mínima em uma árvore de busca binária. Escreva uma função “max” que encontre uma chave máxima.

Exercícios propostos

Exercício 6 Escreva uma função para remover elementos de uma árvore binária de busca. Teste todos os 3 casos descritos na aula.